



Faculty of Engineering and Technology
Electrical and Computer Engineering Department

ENCS4380

HW#3 Arduino Part 1

Prepared by

Aya Dahbour 1201738

Section number: 1

Q1) Based on What we did in class Using Timer Library, do the same functions as in Examples 4-6 without using the Timer Library and Delay function.

➤ Example 4:

```
1  const int outputPin = 13; // Change to desired pin
2  unsigned long startTime = 0;
3  long pulseDuration = 600000; // 10 minutes
4
5  void setup() {
6      pinMode(outputPin, OUTPUT);
7      digitalWrite(outputPin, HIGH); // Set initial state of the output pin to HIGH
8      startTime = millis(); // Record start time
9  }
10
11 void loop() {
12     unsigned long currentMillis = millis(); // Get the current time
13
14     if (currentMillis - startTime <= pulseDuration) {
15         // If the elapsed time is less than or equal to the pulse duration, keep the output pin HIGH
16         digitalWrite(outputPin, HIGH);
17     } else {
18         // If the elapsed time exceeds the pulse duration, set the output pin LOW
19         digitalWrite(outputPin, LOW);
20     }
21 }
```

➤ Example 5:

```
1  const int outputPin = 13;
2  const int analogInputPin = A0;
3  unsigned long previousLedMillis = 0;
4  unsigned long previousReadingMillis = 0;
5  unsigned long ledInterval = 100; // LED interval in milliseconds
6  unsigned long readingInterval = 1000; // Reading interval in milliseconds
7
8  void setup() {
9      Serial.begin(9600);
10     pinMode(outputPin, OUTPUT);
11 }
12
13 void loop() {
14     unsigned long currentMillis = millis();
15
16     // Flash output pin at a specific interval
17     if (currentMillis - previousLedMillis >= ledInterval) {
18         previousLedMillis = currentMillis;
19         digitalWrite(outputPin, !digitalRead(outputPin)); // Toggle output state
20     }
21
22     // Read analog input and display result at a specific interval
23     if (currentMillis - previousReadingMillis >= readingInterval) {
24         previousReadingMillis = currentMillis;
25         int sensorValue = analogRead(analogInputPin);
26         Serial.println(sensorValue);
27     }
28 }
```

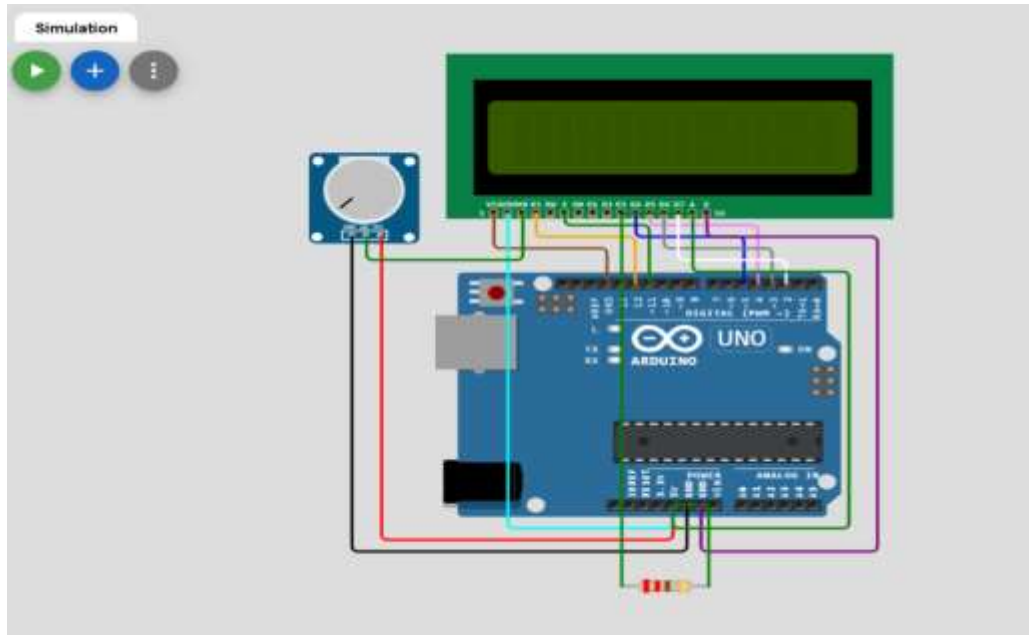
➤ Example 6:

```
1  const int outputPin = 13;
2  const int analogInputPin = A0;
3  unsigned long previousLedMillis = 0;
4  unsigned long previousTickEventMillis = 0;
5  unsigned long ledInterval = 50; // LED interval in milliseconds
6  unsigned long tickEventInterval = 2000;
7  unsigned long additionalActionsTime = 8000;
8  bool additionalActionsDone = false;
9  int slowFlashCounter = 0;
10
11 void setup() {
12     Serial.begin(9600);
13     pinMode(outputPin, OUTPUT);
14 }
15
16 void loop() {
17     unsigned long currentMillis = millis();
18
19     // Flash output pin at a specific interval
20     if (currentMillis - previousLedMillis >= ledInterval) {
21         previousLedMillis = currentMillis;
22         digitalWrite(outputPin, !digitalRead(outputPin)); // Toggle output state
23     }
24
25     // Display the millis on the serial every 2 seconds
26     if (currentMillis - previousTickEventMillis >= tickEventInterval) {
27         previousTickEventMillis = currentMillis;
28         Serial.print("2 second tick: millis ");
29         Serial.println(millis());
30     }
31
32     // Perform additional actions after 8 seconds
33     if (currentMillis >= additionalActionsTime && !additionalActionsDone) {
34         additionalActions();
35         additionalActionsDone = true;
36     }
37
38     // Slowly flash the output pin for five times after 8 seconds
39     if (additionalActionsDone && slowFlashCounter < 5) {
40         static unsigned long slowFlashMillis = 0;
41         unsigned long slowFlashInterval = 1000; // Interval for slow flashing
42         if (currentMillis - slowFlashMillis >= slowFlashInterval) {
43             slowFlashMillis = currentMillis;
44             digitalWrite(outputPin, !digitalRead(outputPin)); // Toggle output state for slow flashing
45             slowFlashCounter++;
46             if (slowFlashCounter == 5) {
47                 // Reset the LED interval and counter after five slow flashes
48                 ledInterval = 50;
49                 slowFlashCounter = 0;
50             }
51         }
52     }
53 }
54
55 void additionalActions() {
56     // Actions to perform after 8 seconds
57     Serial.println("Additional actions after 8 seconds");
58     // For example, slow down output pin flashing
59     ledInterval = 500; // Change output pin flashing interval to 500ms
60 }
```

Q2) In this question I built Arduino sketch without using the LiquidCrystal.h library to display text on a 16x2 LCD. It first shows "ENC4380: 2023" on the first row and "My Name &ID" on the second row. After a 5-second delay, it updates the display with "The mission is DONE" scrolling the text from left to right on the second row. This loop repeats indefinitely.

➤ As showed in the figures below (using wokwi online website):

- The Simulation:



- The code:

```
1 // Define LCD pin connections
2 const int lcdRS = 12, lcdEN = 11, lcdD4 = 5, lcdD5 = 4, lcdD6 = 3, lcdD7 = 2;
3
4 void writeToLCD(uint8_t value, bool mode) {
5     digitalWrite(lcdRS, mode); // RS = 0 for command, 1 for data
6
7     // High nibble
8     digitalWrite(lcdD4, (value >> 4) & 0x01);
9     digitalWrite(lcdD5, (value >> 5) & 0x01);
10    digitalWrite(lcdD6, (value >> 6) & 0x01);
11    digitalWrite(lcdD7, (value >> 7) & 0x01);
12    pulseEnable();
13
14    // Low nibble
15    digitalWrite(lcdD4, value & 0x01);
16    digitalWrite(lcdD5, (value >> 1) & 0x01);
17    digitalWrite(lcdD6, (value >> 2) & 0x01);
18    digitalWrite(lcdD7, (value >> 3) & 0x01);
19    pulseEnable();
20 }
21
22 void pulseEnable() {
23     digitalWrite(lcdEN, LOW);
24     delayMicroseconds(1); // Enable pulse must be >450ns
25     digitalWrite(lcdEN, HIGH);
26     delayMicroseconds(1); // Enable pulse must be >450ns
27     digitalWrite(lcdEN, LOW);
28     delayMicroseconds(100); // Commands need >37us to settle
29 }
30
```

```

31 void lcdCommand(uint8_t command) {
32     writeToLCD(command, LOW);
33 }
34
35 void lcdWriteChar(char c) {
36     writeToLCD(c, HIGH);
37 }
38
39 void lcdInit() {
40     pinMode(lcdRS, OUTPUT);
41     pinMode(lcdEN, OUTPUT);
42     pinMode(lcdD4, OUTPUT);
43     pinMode(lcdD5, OUTPUT);
44     pinMode(lcdD6, OUTPUT);
45     pinMode(lcdD7, OUTPUT);
46
47     delay(50); // Wait for more than 40ms after VCC rises to 2.7V
48
49     // Initialize in 4-bit mode
50     lcdCommand(0x03);
51     delay(5); // Wait for more than 4.1ms
52     lcdCommand(0x03);
53     delayMicroseconds(150); // Wait for more than 100us
54     lcdCommand(0x03);
55     lcdCommand(0x02); // Set to 4-bit mode
56
57     // Function Set
58     lcdCommand(0x28); // 4-bit mode, 2 line, 5x8 dots
59     lcdCommand(0x0C); // Display ON, Cursor OFF, Blink OFF
60     lcdCommand(0x06); // Entry mode, increment cursor position, no display shift
61     lcdCommand(0x01); // Clear display
62 }
63
64 void lcdSetCursor(int col, int row) {
65     int rowOffsets[] = { 0x00, 0x40 };
66     lcdCommand(0x80 | (col + rowOffsets[row]));
67 }
68
69 void setup() {
70     lcdInit();
71
72     // Display initial text
73     lcdSetCursor(0, 0);
74     printToLCD("ENC54380: 2023");
75
76     lcdSetCursor(0, 1);
77     printToLCD("Aya Dah1201738");
78
79     delay(5000); // Wait for 5 seconds
80 }
81
82 void loop() {
83     // Clear display
84     lcdCommand(0x01);
85
86     const char *message = "the mission is DONE";
87     int msgLength = strlen(message);
88
89     // Scroll the text from right to left, starting from the second line
90     for (int pos = -16; pos < msgLength; ++pos) {
91         for (int i = 0; i < msgLength; ++i) {
92             int charPos = pos + i; // Calculate position of each character
93
94             // If the character is within the bounds of the display on the second line
95             if (charPos >= 0 && charPos < 16) {

```

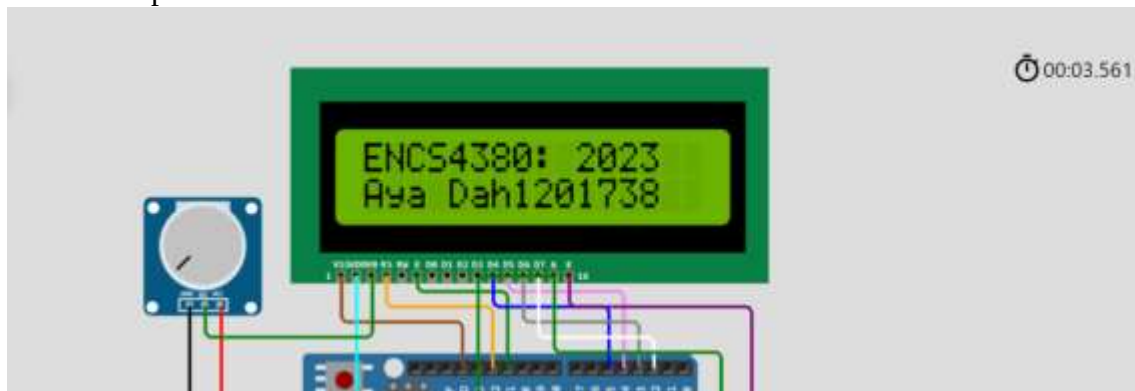


```

96         lcdSetCursor(charPos, 1); // Set cursor for the second line
97         lcdWriteChar(message[i]);
98     }
99
100    // If the character moves to the first line
101    if (charPos >= -16 && charPos < 0) {
102        lcdSetCursor(16 + charPos, 0); // Set cursor for the first line
103        lcdWriteChar(message[i]);
104    }
105    }
106
107    delay(500); // Scroll speed
108    lcdCommand(0x01); // Clear display for the next iteration
109    }
110
111    delay(5000); // Wait for 5 seconds before repeating
112    }
113
114    void printToLCD(const char *string){
115        while (*string) {
116            lcdWriteChar(*string++);
117        }
118    }

```

- The output:



- After 5 second it clears the screen then print “the mission is done” and moving the sentence as follow:



